음성 AI 전문가 과정 · Day 29 / 40

Day 29

LiveKit Agents 기초
Room · Agent Worker · Plugin · 웹 클라이언트

```
server = AgentServer()

@server.rtc_session()
async def entrypoint(ctx: JobContext):
    session = AgentSession(stt=WhisperSTT(),
        llm=make_llm(), tts=KoTTS(), vad=VAD)
    await session.start(agent=Hana(),
                        room=ctx.room)

# 같은 비서, 새 틀: 방이 생기면 출근한다
```

출입증 (JWT) → Agent · AgentSession → STT · LLM · TTS 플러그인 →
AgentServer · entrypoint → 진짜 방 · 브라우저
6 주차 M6 · 4 시간 · livekit-agents 1.8.10 · Edge TTS

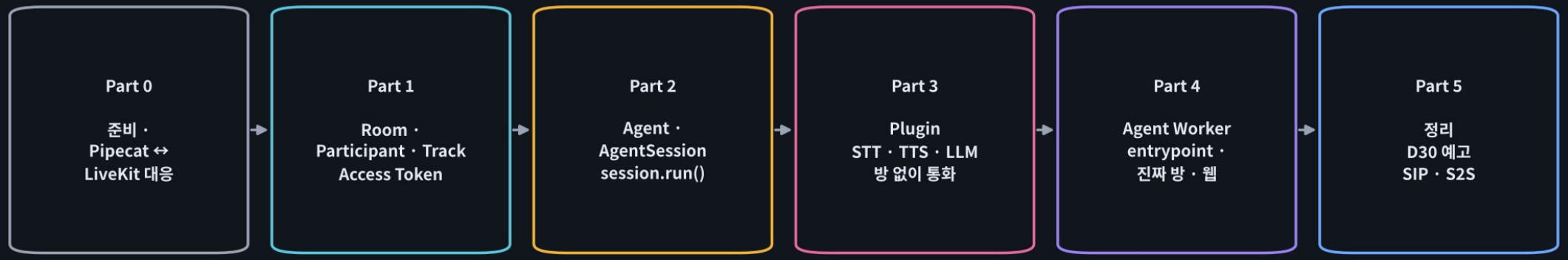
0.0 · 오늘 답할 질문

비서를 진짜 통화 서비스로 띄우려면

1	사용자와 비서는 어디서 , 어떻게 만나나 ?	Part 1
2	방에 들어가는 출입증은 누가 만들고 , 왜 못 고치나 ?	Part 1
3	비서의 " 역할 " 과 " 돌리는 장치 " 는 어떻게 나뉘나 ?	Part 2
4	마이크 없이 대화를 먼저 시험하려면 ?	Part 2
5	내가 가진 Whisper · Edge TTS · Qwen 을 LiveKit 에 꽂으려면 ?	Part 3
6	방이 생기면 누가 비서를 데려오나 ?	Part 4

방 · 출입증 → 대본 · 진행자 → 플러그인 → 작업자

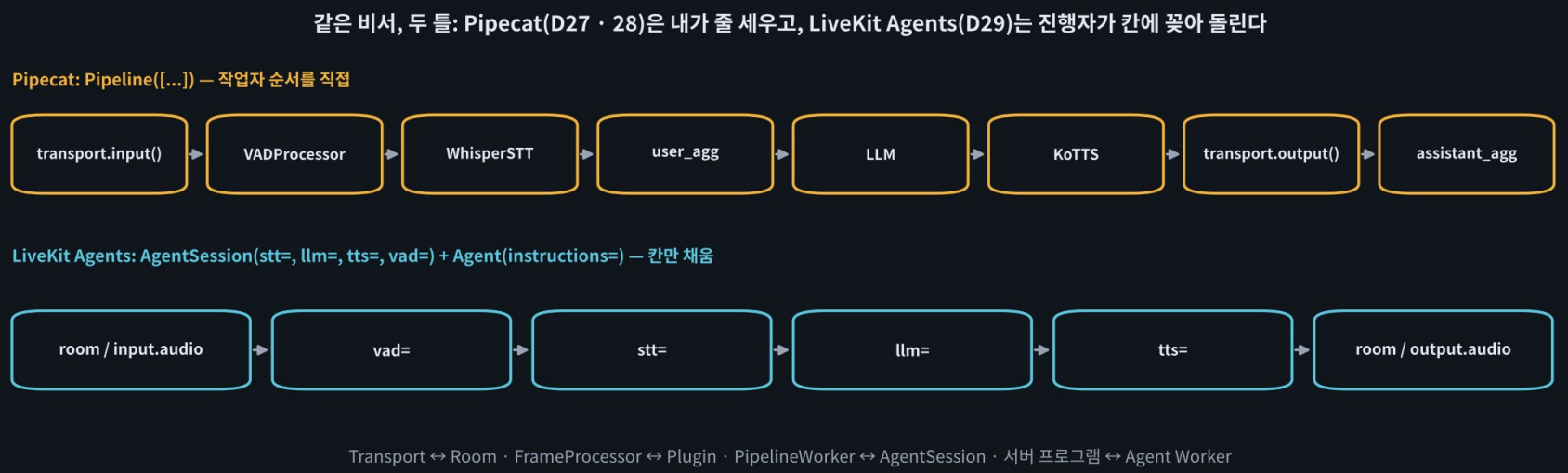
D29 · 같은 비서를 LiveKit 틀에: 방 · 출입증 → 대본 · 진행자 → 플러그인 → 작업자 · 진짜 방



모델은 그대로 (Whisper · Qwen3/클라우드 LLM · Edge TTS · Silero VAD) — 바뀌는 것은 '엮는 틀'과 '소리가 오가는 길'

교시	내용	빈칸	애니메이션
1 교시	Part 0 ~ 1 대응표 · Room · Participant · Track · Access Token	1 · 2	D29-1 · 2 · 3
2 교시	Part 2 Agent · AgentSession · LLM 플러그인 · session.run	3 · 4	D29-4
3 교시	Part 3 STT · TTS 플러그인 · 방 없이 통화	5 · 6 · 7	D29-5 · 6
4 교시	Part 4 ~ 5 Worker · fake job · 진짜 방 · 웹 · 정리	8 · 9	D29-7 · 8 · 9

Pipecat 에서 LiveKit 으로 : 같은 일 , 다른 이름



Transport ↔ Room · FrameProcessor ↔ Plugin · PipelineWorker ↔ AgentSession · 서버 프로그램 ↔ Agent Worker

하는 일	Pipecat (D27 · 28)	LiveKit Agents (오늘)
엮어 돌리기	`Pipeline + PipelineWorker`	`AgentSession`
지시문	`LLMContext system`	`Agent(instructions=…)`
내 STT · TTS	`run_stt · run_tts`	`_recognize_impl · ChunkedStream._run`
소리 길	`Transport`	`Room (WebRTC)`
서비스로	직접 서버	`Agent Worker + Job`

비서도 방에 들어오는 참가자 한 명

Room 하나 = 참가자들이 트랙을 publish하고 subscribe하는 곳 · LiveKit 서버(SFU)는 골라서 전달만



들어갈 때마다 출입증(Access Token, JWT)이 필요 — 누가(identity) · 어느 방(room) · 무엇을(publish / subscribe)

🍎 화상회의 방

회의실 (Room) 에 사람 (Participant) 이 들어와 각자 마이크 (Track) 를 켠다 . 서버는 내 소리를 다른 사람에게 나눠 줄 뿐 . AI 비서는 참석자로 들어오는 프로그램 .

이름	뜻
`publish`	내 트랙 내보내기
`subscribe`	남의 트랙 받기
`kind = agent`	AI 비서 참가자

▶ 애니메이션 D29-2 방 · 참가자 · 트랙

출입증 = 누가 · 어느 방 · 무엇을 + Secret 서명

```
grants = api.VideoGrants(room_join=True, room=room,
                         can_publish=True, can_subscribe=True)
t = (api.AccessToken(DEV_KEY, DEV_SECRET)
     .with_identity("minjun")
     .with_grants(grants)
     .with_ttl(datetime.timedelta(minutes=10)))
t = t.with_kind("agent")      #  비서일 때만
jwt = t.to_jwt()              #  머리 . 내용 . 서명
```

🧩 **빈칸 1 · 2**

room_join (방에 들어가는 권한)

to_jwt() (서명해서 글자 한 줄로)

참가자	kind	publish
사용자	standard	True
비서	agent	True
참관인	standard	False

1. 백엔드가 Secret 으로 출입증 발급 → 브라우저는 완성된 토큰만 받음
2. 서버는 서명을 다시 계산해 비교 → 한 글자라도 고치면 거절

누구나 읽지만 , 아무나 고칠 수는 없다

```
eyJhbGciOiJIUzI1NiIs… . eyJzdWIiOiJtaW5qdW4iLCJ2aWRlbyI6… . XHfSCdI0_EXlymmmG87o…
```

결과	서버 검증
① 원래 토큰	통과 ✔️
② 내용을 고친 토큰 (roomAdmin 추가)	거절 ✖️ InvalidSignatureError
③ 다른 Secret 으로 확인	거절 ✖️ InvalidSignatureError

📐 핵심 식 하나

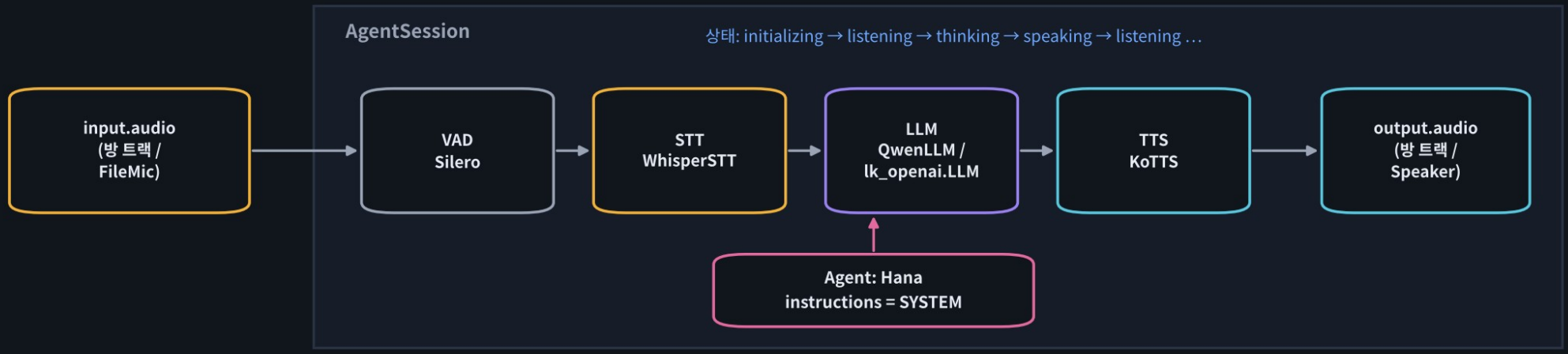
서명 = HMAC-SHA256(Secret, 머리 + "." + 내용)

❓ **출입증을 브라우저 JavaScript 가 만들면 ? → Secret 이 노출 → 누구나 관리자 토큰 . 반드시 백엔드에서 , ttl 은 짧게 .**

▶ 애니메이션 D29-3 토큰의 세 조각

Agent = 무엇을 (대본) · AgentSession = 어떻게 (진행자)

Agent = 무엇을 (대본) · AgentSession = 어떻게 (귀 · 머리 · 입을 엮는 진행자)



`await session.start(agent=Hana(), room=ctx.room)` · `session.run(user_input=...)` 는 STT · TTS 없이 LLM 칸만으로 한 턴

🍎 **콜센터**

Agent 는 상담원의 업무 매뉴얼 , AgentSession 은 전화 연결 · 받아쓰기 · 말하기를 해 주는 상담 시스템 .

▶ 애니메이션 D29-4 Agent와 AgentSession

클라우드는 기성 플러그인, 로컬은 직접 만든 플러그인

```python
class QwenLLM(lk_llm.LLM):                  # 규칙 1
    def chat(self, *, chat_ctx, tools=None, ...):
        return QwenStream(self, chat_ctx=chat_ctx, ...)

class QwenStream(lk_llm.LLMStream):         # 규칙 2
    async def _run(self):
        messages, _ = self._chat_ctx.to_provider_format("openai")
        it = qwen_stream(messages)
        while (piece := await asyncio.to_thread(next, it, None)):
            self._event_ch.send_nowait(lk_llm.ChatChunk(
                id=rid, delta=lk_llm.ChoiceDelta(content=piece)))
```

```python
# 클라우드 (키가 있을 때)
lk_openai.LLM(
  model=MODELS[p],
  api_key=secret(KEYS[p]),
  base_url=BASE_URLS[p])
```

💬 **쉽게 말하면** D27 의 LLMTextFrame push = 오늘의 ChatChunk 보내기

Agent(instructions=...) · session.run(user_input=...)

```
class Hana(Agent):
    def __init__(self):
        super().__init__(instructions=SYSTEM)

session = AgentSession(llm=make_llm())
await session.start(agent=Hana())
result = await session.run(user_input=q)
ev = result.expect.next_event().is_message(role="assistant")
print(ev.event().item.text_content)
```

🧩 **빈칸 3 · 4**

instructions (시스템 프롬프트)

run (한 턴 돌리고 RunResult)

결과 (예)	값
턴	3
세션이 기억한 메시지	6
LLM TTFT	0.1 ~ 0.8 초

💬 **무대에 올리기 전 대본 리딩 : 귀 (STT) 탓인지 머리 (LLM) 탓인지 나눠 본다**

음성 비서도 자동 시험 (pytest) 으로

시험 줄	뜻
`result.expect.next_event().is_message(role="assistant")`	비서가 반드시 답해야 함
`result.expect.next_event(type="function_call")`	여기서는 도구 (예약) 를 불러야 함 (D30)
`result.expect.skip_next_event_if(type="message")`	있으면 건너뜀
`session.history.items`	세션이 쌓은 대화 기록

1. 프롬프트 · 모델을 바꿀 때마다 같은 대화 30 개를 글자로 자동 시험
2. 통과하면 그다음에 음성 (가짜 마이크 → 진짜 방) 으로
3. 세션은 history 를 통째로 보냄 → 통화가 길수록 입력 토큰 · 지연 증가

정해진 부모 클래스 + 메서드 하나

플러그인 = 정해진 부모 클래스 + 메서드 하나 · 나머지(재시도 · 지연 측정 · 쪼개기)는 부모와 세션이

칸	부모 클래스	우리가 채울 곳	돌려주는 것	세션(StreamAdapter)이 대신
STT	stt.STT	**_recognize_impl(buffer)**	SpeechEvent(FINAL_TRANSCRIPT)	VAD로 한 마디씩 잘라 줌
LLM	llm.LLM → LLMStream	**chat() → _run()**	ChatChunk 조각들	대화 기록 · 지시문을 모아 줌
TTS	tts.TTS → ChunkedStream	**synthesize() → _run(emitter)**	emitter.push(PCM 바이트)	문장 단위로 모아 넣어 줌

capabilities(streaming=False) 라고 신고하면 → 세션이 StreamAdapter로 감싸서 '흐르는 통화'에 맞춰 줌

▶ 애니메이션 D29-6 플러그인의 콘센트

_recognize_impl → SpeechEvent(FINAL_TRANSCRIPT)

```
class WhisperSTT(lk_stt.STT):
    def __init__(self):
        super().__init__(capabilities=lk_stt.STTCapabilities(
            streaming=False, interim_results=False))
    async def _recognize_impl(self, buffer, *, language=None, ...):
        frame = rtc.combine_audio_frames(buffer)
        y = to_float(frame)  # 48 kHz → 16 kHz
        text = await asyncio.to_thread(stt, y)
        return lk_stt.SpeechEvent(
            type=lk_stt.SpeechEventType.FINAL_TRANSCRIPT,
            alternatives=[lk_stt.SpeechData(language="ko", text=text)])
```

🧩 빈칸 5·6

_recognize_impl (우리가 채우는 자리)

FINAL_TRANSCRIPT (확정)

⚠ recognize 를 덮어쓰면 재시도 · metrics 가 사라짐

사건	뜻
INTERIM	중간 받아쓰기
FINAL	확정
END_OF_SPEECH	말 끝

synthesize() → ChunkedStream._run(output_emitter)

```
class KoTTS(lk_tts.TTS):
    def __init__(self):
        super().__init__(capabilities=lk_tts.TTSCapabilities(streaming=False),
                         sample_rate=SR, num_channels=1)
    def synthesize(self, text, *, conn_options=...):
        return KoChunk(tts=self, input_text=text, conn_options=conn_options)

class KoChunk(lk_tts.ChunkedStream):
    async def _run(self, output_emitter):
        y = await asyncio.to_thread(tts, normalize_ko(self.input_text))
        output_emitter.initialize(..., sample_rate=SR, mime_type="audio/pcm")
        output_emitter.push(pcm)
        output_emitter.flush()
```

🧩 빈칸 7

push (소리 바이트 밀어 넣기)

1. initialize: 16 kHz · 모노 · PCM
2. push: 소리 바이트
3. flush: 이 문장 끝

StreamAdapter + 가짜 마이크 · 스피커로 통화 전체 시험

	들어오는 것	StreamAdapter
STT	끝없는 마이크 소리	VAD 로 한 마디씩
TTS	LLM 글자 조각	문장 단위로

자리	방에서	오늘
`session.input.audio`	사용자 마이크 트랙	FileMic
`session.output.audio`	비서 목소리 트랙	Speaker

📐 **핵심 식 하나**

e2e_latency ≈ 턴 끝 판단 + LLM TTFT + 첫 문장 + TTS TTFB

메시지마다 metrics: transcription_delay · end_of_turn_delay · llm_node_ttft · tts_node_ttfb · e2e_latency

▶ 애니메이션 D29-5 세션 안의 흐름

💬 **쉽게 말하면** 서버 없이도 통화 전체를 시험할 수 있다 → 실무에서 가장 많이 쓰는 방법

방이 생기면 비서가 출근한다

Agent Worker의 흐름: 등록 → 방 생김 → 일감(Job) → entrypoint(ctx) → session.start(room=ctx.room)



🍎 콜센터 상담원

출근해 대기 명단에 등록 → 전화가 오면 배정 → 통화 . 한 상담원이 여러 통화도 .

▶ 애니메이션 D29-7 작업자의 하루

@server.rtc_session() · session.start(room=ctx.room)

```
server = AgentServer(job_executor_type=JobExecutorType.THREAD)
@server.rtc_session()
async def entrypoint(ctx: JobContext):
    await ctx.connect()
    session = make_session()
    if ctx.is_fake_job():
        attach_fake_io(session, [VOICE1, VOICE2], t0)
    await session.start(agent=Hana(), room=ctx.room)
    session.say(GREET)

asyncio.ensure_future(server.run(devmode=True, unregistered=True))
await server.simulate_job("hanbit-fake-room", fake_job=True)
```

🧩 빈칸 8·9

rtc_session (방 통화 일감 등록)

ctx.room (배정된 방)

방 이름이 console 이면 fake job (정상)

Colab 안 LiveKit 서버 + Python 참가자

```
!curl -sSL https://get.livekit.io | bash
subprocess.Popen(["livekit-server", "--dev", "--bind", "127.0.0.1"])

room_server = AgentServer(ws_url=DEV_URL,
    api_key=DEV_KEY, api_secret=DEV_SECRET, ...)
room_server.rtc_session()(entrypoint)
asyncio.ensure_future(room_server.run(devmode=True))

room = rtc.Room()
await room.connect(DEV_URL, make_token("minjun", room=ROOM2))
track = rtc.LocalAudioTrack.create_audio_track("mic", source)
await room.local_participant.publish_track(track, ...)
```

누가	무엇
LiveKit 서버	SFU (이 Colab)
작업자	같은 entrypoint
사용자	Python 참가자
비서 상태	lk.agent.state 속성

⚠ 브라우저는 Colab 안 서버에 못 들어옴 → 4-3 Cloud

▶ 애니메이션 D29-8 세 가지 연결

브라우저 마이크로 통화 : LiveKit Cloud + jupyter.run_app

```python
from livekit.agents import jupyter
cloud_server = AgentServer(ws_url=URL,
    api_key=KEY, api_secret=SECRET)
cloud_server.rtc_session()(entrypoint)
jupyter.run_app(cloud_server)
# 1분 토큰 · 셀 위젯 · 작업자 + 일감
```

```javascript
const { url, token } = await fetch(
  "/api/livekit-token").then(r => r.json());
const room = new Room();
room.on(RoomEvent.TrackSubscribed, t =>
  t.kind === "audio" && document.body
    .appendChild(t.attach()));
await room.connect(url, token);
await room.localParticipant
  .setMicrophoneEnabled(true);
```

연결	서버	사용자	필요한 것
fake job (4-1)	없음	가짜 마이크 · 스피커	없음
Colab 안 서버 (4-2)	livekit-server --dev	Python 참가자	없음
LiveKit Cloud (4-3)	Cloud	브라우저 마이크	Cloud 키 3 개
Agents Playground	Cloud	웹 플레이그라운드	Cloud 키 + 실행 중 작업자

출입증으로 들어가고 , 대본 + 진행자로 , 부품을 꽂아 , 작업자가 부른다

개념	코드
출입증	`VideoGrants(room_join=True) · to_jwt()`
대본	`Agent(instructions=SYSTEM)`
글자 시험	`session.run(user_input=q)`
STT 플러그인	`_recognize_impl → FINAL_TRANSCRIPT`
TTS 플러그인	`output_emitter.push(pcm)`
작업자	`@server.rtc_session()`
방에 올리기	`session.start(agent, room=ctx.room)`

다음 : D30 LiveKit 심화 & S2S

SIP 로 진짜 전화를 방에 연결하고 , 예약 → 수납 Agent 로 Handoff, 귀 · 머리 · 입을 한 모델 (OpenAI Realtime · Gemini Live) 로 바꿔 무엇이 빨라지고 무엇을 잃는지 잽니다 .

`worksheet.html` 40 문항

개념 워크시트 · 노션 제출

`animation.html` 9 개 탭

인터랙티브 애니메이션

`D29_LiveKit 기초 _ 실습 .ipynb` 빈칸 9

실습 노트북 · 키 선택

온라인 자료 imguru-mooc.github.io/AI_Voice/day29